

Galana fuel card integration: API test guide and flows

Base URL: `https://<site>/api/method/jpl_fuel_card.api.partner.v1.<name>` (on the dev box: `http://127.0.0.1:8000` with Host: `crm`).

Every response is wrapped as `{"message": ...}`. Login returns `{"user", "tokens", "message"}` at the top level. Errors return `{"exception": "...", "_server_messages": "...}"` with a 4xx or 5xx status. The readable text is inside `_server_messages`.

1. Accounts and roles

Account	Frappe role	Can call
Galana service account (<code>galana.partner.api@galana.co.ke</code>)	Galana Partner API	Everything
Jaguar (Brian)	Jaguar Partner API	<code>login</code> , <code>refresh_token</code> , <code>logout</code> , <code>customer_float</code> , <code>epra_prices</code> , <code>transaction_events</code>
POS terminal	POS Partner API	<code>login</code> , <code>refresh_token</code> , <code>logout</code> , <code>ticket_status</code> , <code>validate_ticket</code> , <code>pos_device</code> , <code>authorise</code> , <code>start_fuelling</code> , <code>request_otp</code> , <code>confirm_otp</code> , <code>cancel_transaction</code> , <code>transaction_events</code> , <code>epra_prices</code> , <code>customer_float</code>
Portal staff	Galana <code><RoleName></code> (e.g. Galana FINANCE_MAKER)	Galana portal endpoints (section 4)

Anything outside an account's list returns **403**. A missing or bad token returns **401**.

2. Authentication

Login (no token needed):

```
curl -X POST "$BASE/login" -H "Content-Type: application/json" \
  -d '{"usr": "<username>", "pwd": "<password>"}'
```

Response:

```
{
  "user": {"name": "...", "full_name": "...", "email": "..."},
  "tokens": {"access_token": "...", "refresh_token": "...", "expires_in": 3600},
  "message": "Logged in successfully"
}
```

Send `Authorization: Bearer <access_token>` on every other call. Tokens expire after 3600 seconds.

Refresh (rotates the tokens; the old one is revoked):

```
curl -X POST "$BASE/refresh_token" -H "Content-Type: application/json" -d '{"refresh_token": "<refresh_token>"}
```

Logout (revokes the access token):

```
curl -X GET "$BASE/logout" -H "Authorization: Bearer <access_token>"
```

Signed POSTs (transaction_events only) also need:

- `X-Client-Id`: the client id from the OAuth Client record.
- `X-Timestamp`: unix seconds. Must be within 300 seconds of server time.
- `X-Signature`: `sha256=` + HMAC-SHA256 of `"<X-Timestamp>." + raw body`, keyed with the client secret.

3. Jaguar endpoints (Brian)

These are the only partner endpoints Jaguar may call.

3.1 customer_float: is there enough float?

```
curl -G "$BASE/customer_float" -H "Authorization: Bearer $T" \
--data-urlencode "customer=Jaguar Petroleum" --data-urlencode "amount=5000"
```

Response:

```
{"message": {"customer": "Jaguar Petroleum", "payment_type": "Prepaid", "account_status": "Active",
"currency": "KES", "as_of": "2026-10-06", "required_amount": 5000.0,
"float_balance": 0.0, "available": 0.0, "sufficient": false}}
```

- Prepaid accounts return `float_balance`. Credit accounts return `credit_limit`, `utilized_credit` and `available_credit`.
- `sufficient` is `true` when `available` covers `amount`.
- Other customers return **404** ("Customer X not found").

3.2 epra_prices: current prices

```
curl -G "$BASE/epra_prices" -H "Authorization: Bearer $T" --data-urlencode "product=Automotive Gasoi
l Local Lot Costed (Litre)"
```

Response:

```
{"message": {"as_of": "2026-10-06", "prices": [
{"product": "Automotive Gasoil Local Lot Costed (Litre)", "price_per_litre": 214.0,
"currency": "KES", "effective_from": "2026-10-06", "effective_to": null}]}}
```

`product` is optional. With no active price the call returns **404**.

3.3 `transaction_events`: push a completed fuelling (signed)

```
BODY='{ "ticketReference": "<ticket name>", "authorizationCode": "12345678", "ptsId": "TEST-POS-0002",
"transactionNumber": "TX-0001", "transactionDateTime": "2026-10-06 10:15:00", "amount": 5000,
"volume": 23.25, "price": 214, "pumpNumber": 1, "nozzleNumber": 2}'
TS=$(date +%s)
SIG="sha256=$(printf '%s.%s' "$TS" "$BODY" | openssl dgst -sha256 -hmac "$CLIENT_SECRET" | awk '{pri
nt $2}')
```

```
curl -X POST "$BASE/transaction_events" -H "Authorization: Bearer $T" -H "Content-Type: application/
json" \
-H "X-Client-Id: $CLIENT_ID" -H "X-Timestamp: $TS" -H "X-Signature: $SIG" -d "$BODY"
```

- Required: `ptsId`, `transactionNumber`, `transactionDateTime`, `amount`, and `ticketReference` or `authorizationCode`.
- Optional: `volume`, `price`, `totalVolume`, `totalAmount`, `pumpNumber`, `nozzleNumber`, `tagNo`, `clockNo`, `shiftNo`, `customerPin`, `customerName`.
- Sending the same `ptsId` and `transactionNumber` again is recognised as a duplicate and is not counted twice.
- A wrong or missing signature returns **401**. A timestamp outside 300 seconds returns **401**.

4. POS endpoints

The POS calls these in order for one fuelling.

Step	Endpoint	Method	Purpose
1	<code>pos_device</code>	GET	Device active, on an allocated station, on approved software
2	<code>validate_ticket</code>	GET	Ticket valid to fuel? Read-only, does not use the ticket
3	<code>authorise</code>	POST	Reserve the fuel (only if the POS requests it)
4	<code>start_fuelling</code>	POST	Ticket moves from Reserved to Dispensing
5	<code>transaction_events</code>	POST	Record the completed fuelling (signed)
-	<code>cancel_transaction</code>	POST	Cancel a ticket that has not started fuelling

Example checks:

```
curl -G "$BASE/pos_device" -H "Authorization: Bearer $T" --data-urlencode "serial_number=TEST-POS-00
02" --data-urlencode "software_version=TEST-1.0"
curl -G "$BASE/validate_ticket" -H "Authorization: Bearer $T" --data-urlencode "authorization_code=1
2345678"
curl -X POST "$BASE/start_fuelling" -H "Authorization: Bearer $T" -H "Content-Type: application/jso
n" \
-d '{"pos_serial": "TEST-POS-0002", "software_version": "TEST-1.0", "authorization_code": "12345678"}'
```

`validate_ticket` response: `{"valid": true, "reason": null, "ticket_reference": "...", "status": "Reserved", "portal_status": "Approved", "customer": "...", "vehicle_number": "...", "fuel_type": "Diesel", "authorised_amount": 5000.0, "remaining_amount": 5000.0, "expires_on": "..."}.`

`pos_device` refusals: "POS device is deactivated", "not allocated to a station", "Software version X is not approved" (403 or 417 depending on the check).

`authorise` takes `customer`, `vehicle_id`, `fuel_amount` and optional `fuel_station`. It returns the 8-digit code, the expiry, the vehicle and the driver. It is refused with "Insufficient float" when the wallet is short.

`cancel_transaction` takes `ticket_reference` or `authorization_code` and a `reason`. Only Reserved tickets can be cancelled.

OTP calls: `request_otp` takes `vehicle_id` and `id_number`. `confirm_otp` takes `otp` and `id_number`.

5. Galana portal endpoints (internal)

These need the Galana service account, and an `actor` (the signed-in user's email) for writes. The portal calls them; they are not for external use.

Area	Endpoints
Customers	<code>galana_customers</code> , <code>portal_customers</code> , <code>portal_customer</code>
Dashboard and reports	<code>portal_dashboard</code> , <code>portal_report</code>
Tickets and transactions	<code>tickets</code> , <code>ticket_status</code> , <code>transactions</code> , <code>transaction</code> , <code>ticket_reconciliation</code> , <code>customer_summary</code> , <code>customer_consumption</code> , <code>audit_trail</code>
Stations, dealers, devices	<code>stations</code> , <code>station</code> , <code>save_station</code> , <code>set_station_status</code> , <code>dealers</code> , <code>dealer</code> , <code>update_dealer</code> , <code>pos_devices</code> , <code>pos_device_record</code> , <code>create_pos_device</code> , <code>set_pos_device_status</code> , <code>fuel_products</code>
Software and EPRA	<code>approved_versions</code> , <code>add_approved_version</code> , <code>remove_approved_version</code> , <code>epra_price_history</code> , <code>add_epra_price</code>
Wallet	<code>topups</code> , <code>create_topup</code> , <code>decide_topup</code>
Adjustments	<code>adjustments</code> , <code>create_adjustment</code> , <code>decide_adjustment</code>
Reversals	<code>reversals</code> , <code>create_reversal</code> , <code>decide_reversal</code>
Settlements	<code>settlements</code> , <code>credit_notes</code>
Users	<code>portal_users</code> , <code>portal_user</code> , <code>create_portal_user</code> , <code>update_portal_user</code> , <code>set_portal_user_status</code> , <code>reset_portal_user_password</code> , <code>portal_login</code> , <code>portal_profile</code>
Settings and audit	<code>portal_settings</code> , <code>save_portal_settings</code> , <code>audit</code> , <code>record_audit</code> , <code>audit_log</code>
Vehicles	<code>vehicle_details</code>

Example, a wallet top-up (maker creates, checker decides):

```
curl -X POST "$BASE/create_topup" -H "Authorization: Bearer $G" -H "Content-Type: application/json" \
  -d '{"actor":"finance.maker@galana.co.ke","customer":"Jaguar Petroleum","amount":25000,
      "funding_account":"KES 1234567890 (Jaguar)","reference":"TEST TOPUP 9"}'
curl -X POST "$BASE/decide_topup" -H "Authorization: Bearer $G" -H "Content-Type: application/json"
```

```
\
-d '{"actor":"finance.checker@galana.co.ke","request_id":"<id>","decision":"Approved"}'
```

Rules enforced on the server: the maker can't decide their own request; a rejection needs a reason; a decided request can't be decided again; the customer must be a Galana customer.

6. Flows

6.1 Fuel ticket lifecycle

```
authorise (Reserved, code issued)
  |-- cancel_transaction --> Refunded (Cancelled)
  |-- expiry time passes --> Expired
  |-- start_fuelling -----> Dispensing (Fuelling)
    |-- transaction_events --> Consumed (Fulfilled), fuelling row Success
```

`validate_ticket` and `ticket_status` only read. Verifying a code in the Jaguar portal does not change its status.

6.2 POS fuelling

```
POS -> pos_device           device active, allocated, approved software?
POS -> validate_ticket      valid? (read-only)
POS -> start_fuelling       Reserved -> Dispensing
POS -> transaction_events   signed; records litres and amount; ticket Consumed
```

6.3 Wallet balance (the one formula)

```
balance = approved top-ups
          + approved credit adjustments
          - approved debit adjustments
          + approved reversals (the fuelling amount goes back)
          - successful fuelling
```

Pending and rejected requests change nothing. Used by `customer_float`, `authorise`, the dashboard and customer pages.

6.4 Approvals (top-ups, adjustments, reversals)

```
Maker creates -> Pending Approval
Checker decides (not the maker)
  Approved -> counts in the balance above
  Rejected -> needs a reason; no effect
```

Each step is written to Activity Log with the user, time, record and outcome.

6.5 Settlements and credit notes

For each completed fuelling:

```
under-canopy discount = litres x under_canopy_discount_per_litre
Jaguar discount       = litres x jaguar_discount_per_litre
net to dealer         = gross - under-canopy discount          (Jaguar discount is NOT deducted)
credit note (UNDER_CANOPY) = under-canopy discount
credit note (CONTRACTUAL)  = Jaguar discount
```

Rates are in Galana Settings and start at 0. Galana Finance (FINANCE_MAKER and FINANCE_CHECKER) sets them; System Administrators can change every setting. Settlements and credit notes are pending. Nothing is posted to any ledger.

6.6 Scope and access in the portal

- Only Galana customers appear anywhere (`galana_customers`, from Galana Settings).
- Dashboard sections depend on role: operations (ops, admin, auditor), finance (finance roles, admin, auditor), Jaguar (Jaguar and the finance and ops roles), dealer (dealer manager).
- A dealer manager sees only the station on their user record (`galana_station`) and no customer accounts.
- Fuelling is attributed to a station through the POS device's current allocation. This assumes `pts_id` is the POS serial number.

7. Frontend (Galana portal) to backend map

Page	Backend call
Dashboard	<code>portal_dashboard</code>
Transactions and detail	<code>transactions</code> , <code>transaction</code>
Failed transactions	<code>transactions</code> with <code>status=FAILED</code>
Fuel tickets and detail	<code>tickets</code> , <code>ticket_status</code>
Reconciliation	<code>ticket_reconciliation</code> (Jaguar feed)
Customers and detail	<code>portal_customers</code> , <code>portal_customer</code>
Funding wallet	<code>topups</code> , <code>create_topup</code> , <code>decide_topup</code>
Adjustments	<code>adjustments</code> , <code>create_adjustment</code> , <code>decide_adjustment</code>
Reversals	<code>reversals</code> , <code>create_reversal</code> , <code>decide_reversal</code>
Settlements	<code>settlements</code>
Credit notes	<code>credit_notes</code>
Stations	<code>stations</code> , <code>save_station</code> , <code>set_station_status</code> , <code>fuel_products</code> , <code>dealers</code>
Dealers	<code>dealers</code> , <code>update_dealer</code>
POS devices	<code>pos_devices</code> , <code>create_pos_device</code> , <code>set_pos_device_status</code> , <code>approved_versions</code>
EPRA prices	<code>epra_price_history</code> , <code>add_epra_price</code>
Reports	<code>portal_report</code>

Page	Backend call
Users	portal_users, create_portal_user, update_portal_user, set_portal_user_status, reset_portal_user_password
Settings	portal_settings, save_portal_settings
Audit log and export	audit_log
Validate ticket (Jaguar check)	validate_ticket

Sign-in uses portal_login (password check and roles) and portal_profile (status on each session check).

8. Test data on `crm`

Created for testing, all named TEST:

- Stations TEST-ST-01, TEST-ST-02 under TEST Dealer.
- POS devices TEST-POS-0001 to 0003.
- Vehicles KTEST 001 to 003.
- 30 completed fuellings and 3 failed attempts, all with notes = "TEST DATA".
- Tickets TEST-TKT-.*.
- Top-ups and adjustments named TEST ...

The wallet is currently overdrawn by this data, so authorise will refuse until it is removed or more is approved.

9. Known gaps

- Jaguar inbound (tickets and customers pushed to Galana) and outbound (Galana calling Jaguar): not built, need Jaguar's contract.
- Idempotency-Key header on transaction_events: duplicates are blocked by POS ID and transaction number, not by the header.
- Reconciliation records and the exception queue: not built, waiting on a decision.
- The Jaguar portal's own balance still reads the old ledger, so it can differ from the partner balance above.
- Driver OTP SMS: authorise returns the code, but nothing sends it.
- Stuck "Dispensing" tickets never expire.
- v1.ingest_fuel_transaction cannot load until the JPL default POS customer is set. Its public access is closed.
- The actor field on portal writes is trusted from the Galana service account. Anyone holding that account can name any user as the actor.